

# YunTAPR-Net

## Phase-B v2 消融候选实现

## 合成验证与研究者代码学习材料

工程候选 / 独立科学审查仍待研究者

2026 年 10 月 10 日 / v1

实际实现与合成验证已完成；正式科学协议和训练仍未批准。

CPU 177 / CUDA 3 合成测试通过，静态检查 11 项；没有模型 forward、optimizer step、原始观测或 checkpoint 读取。本轮开发委托替代上轮候选阶段的练习先行顺序，但不接受 Phase-A 科学结果或追认历史 B1 恢复。

### 目录

|                            |    |
|----------------------------|----|
| 1 候选实现架构与责任边界              | 2  |
| 2 闭合消融配置与运行身份合同            | 3  |
| 3 张量契约与工程保护解读              | 4  |
| 4 Focal 候选实现逐段解读           | 5  |
| 5 Conditional Pinball 逐段解读 | 6  |
| 6 总损失、监督张量与指标分母            | 7  |
| 7 未加权评价接口解读                | 8  |
| 8 E0 冻结数学兼容性：实际合成验证        | 9  |
| 9 合成梯度与环境验证报告              | 10 |
| 10 独立 v2 runner 的非执行型集成提案  | 11 |
| 11 元数据审核与阻断 runner 逐段解读    | 12 |
| 12 研究者代码与科学审查清单（未批准）       | 13 |
| 13 研究者代码学习手册与注释源码副本        | 14 |
| 14 关键数学、解析梯度与源码对应          | 26 |

## 1 候选实现架构与责任边界

科学定义来自冻结 Focal、Pinball、total\_loss、v2 quantile 输出守卫及上轮 E0/E1/E2 提案；工程保护在隔离目录内补齐。当前委托允许候选代码和合成 autograd，未批准冻结合同修改、数据 preflight、真实模型推理或 optimizer 更新。

| 模块            | 输入/输出                                       | 责任                                            |
|---------------|---------------------------------------------|-----------------------------------------------|
| config.py     | E0/E1/E2 → 不可变参数；model/seed → RunSpec       | 闭合集合、18 身份、纯数学预算与 LR 前缀                       |
| validation.py | logit/qlog/rate/独立 bool masks → Supervision | 形状、设备、dtype、finite、支撑/严格序、float32 标签与计数       |
| focal.py      | logit、硬标签、valid、显式 alpha/gamma → S_occ      | 稳定 BCE、类别权重、focusing、有效位置 sum                 |
| pinball.py    | FP64 qlog/log_target/rainy_valid → S_qr     | 固定 32 tau、mean 后 sum，不转换物理域                   |
| total.py      | 两头合成张量与固定 config → CandidateLossResult      | 唯一权重、N_valid 分母、零监督停止                         |
| metrics.py    | 候选 result / 同输入张量 → ReportingSums           | unweighted CPB、全局 pool、固定 E0 FP64 common core |
| readiness.py  | 合成或声明的 SHA 元数据 → 审查结果                       | fresh/恢复绑定保护；BlockedRunner 永远抛错               |
| __init__.py   | 公开接口导入                                      | 没有数据或训练启动行为；导入损失 API 会加载 torch                |

调用顺序：candidate\_loss → supervision → occurrence\_numerator 与 conditional\_pinball\_numerator → combine\_numerators → result。Pinball 的 \_pinball\_from\_errors 是私有算术内核，专供已经定义的 signed errors；不是绕过 qlog 资格的正式入口。P1-P4 用内核核对手算，不把常数 qlog 伪装成 v2 单调输出。

模型输出仍为 occurrence logit 与 FP64 32 条件分位数，本轮没有实现或修改 head、allocation/span 参数化、tau、epsilon、SP04 和 causal slots。validate\_log\_quantiles 是唯一正式输出守卫复用；没有调用模型构造器、Contract.load、torch.load、数据模块或旧训练入口。该复用保持冻结的 finite、 $q1 > \log_{1p}(.1)$ 、严格递增定义。

supervision 拒绝零有效像元而非返回 batch\_skipped；这与旧正式 runner 对 skipped batch 的最终拒绝一致，但错误提前到候选 loss 入口。有效但全无雨仍保留 occurrence 与 q 的零梯度连接。输入 rate 必须是原 float32 参考，拒绝 requires\_grad；输出全域 finite，即使像元无效也不能掩盖坏模型输出。仅 invalid reference 的临时计算副本置零，源张量不改写。

FP16/BF16 logits 显式提升为 FP32 做 BCE；FP64 logits 保持 FP64 用于梯度差分。qlog/log\_target/tau 固定 FP64。autocast(enabled=False) 防止外层混合精度改写关键路径，转换的反向链仍会将梯度送回原 logits。该输入张量层测试不宣称真实 BF16 模型已通过。

静态工具不 import 候选模块，仅读源码 AST、提案和发表的元数据 SHA。readiness.py 自身只用标准库，但通过公开包导入时 \_\_init\_\_ 会加载 torch；这仍没有实例化模型。真正只需标准库的入口为 scripts/.../static\_review.py。

## 2 闭合消融配置与运行身份合同

config.py 定义私有 `_PARAMETERS` 字典: `E0=(.5,2,1)`, `E1=(.5,0,1)`, `E2=(.5,2,2)`。 `get_config(experiment_id)` 只接收精确字符串 `E0/E1/E2`; 大小写错误、未知 ID、`E3/E4/FinalFit`、`None` 或 `dict` 全部拒绝。它不读取外部配置文件、不选择最优条件。

`AblationConfig` 是 `@dataclass(frozen=True)`: 自动生成构造、比较等方法, 字段是 `experiment_id`、`alpha`、`gamma`、`lambda_q`。 `__post_init__` 在构造后检查类型、`finite` 和 ID 对应精确数值, 禁止 `E0` 名下悄悄使用 `gamma0`。 `frozen` 禁止普通属性赋值; 候选调用前仍重验, 不能把 Python 级防篡改当人类授权。

`finite_number(name,value)` 返回 Python float 但不盲目把字符串或 bool 转数; 判断 type 不是 `isinstance` 用于排除 bool。无默认 `alpha/gamma`, 裸 loss 显式传参; 科学臂只使用 `get_config` 返回值。科学状态 `PROPOSED_FOR_RESEARCHER_APPROVAL` 是常量说明, 不存在升级 `APPROVED` 的 setter。

`RunSpec(experiment_id,model,seed)` 要求模型精确 `B0_MATCHED_V2/B1_V2`, `seed` 为真正 int 且属于 2026/2027/2028。 `run_id` property 动态构造条件、模型与 `seed` 身份; `stage` property 将 2026 归阶段 1、另两 `seed` 归阶段 2, 只是划分, 不是执行批准。 `proposed_runs` 依 `seed→arm→model` 顺序生成 tuple18 项, tuple 不可追加; 同一 `seed` 内条件必须共享样本、初始化和排列。

`workload()` 只做整数 ceil 预算: `10455/physicalbatch2→5228` 更新每 epoch, 尾 `batch1`; `9epoch47052`; 全 `18→846936`。 `10501/valbatch8→1313`, 尾 `batch5`; 每 epoch 验证, 共 212706。阶段 1 六 `run282312`, 阶段 2 十二 `run564624`。没有运行这些更新。新预算测试与旧 `budget_summary` 核对, 不重跑旧科学验收。

`learning_rate_prefix(update)` 只接收 `1..47052` 的真正 int; `W5228`、`U261400`, 不把 `9epoch` 压缩成 cosine 全周期。前 `W` 步线性 warmup 低于 `1e-6` 可以合法; `minLR` 只作用于后续 cosine 段。测试逐一核对全部 `47052` 位置与原 `lr_for_update(steps_per_epoch=5228)` 的相同返回值, 原函数通过 AST 提取纯数学节点调用, 未加载其数据/授权模块。

候选配置仍依赖 `D1/Q1/N0/I3/B9/S0/V0` 的独立正式批准。数据产品、`M1` 配对、原 `float32>.1` 标签、`32tau`、`epsilon1e-4`、`FP64` 守卫、`batch2/accum1/no_drop`、`AdamW` 与 `clip`、逐 epoch 验证及 epoch9 终点不能因为代码 PASS 而改变。未来 runner 须验证机读提案、代码 SHA 及真实授权, 而不是只检查实验 ID 存在。

### 3 张量契约与工程保护解读

validation.py 集中拒绝错误输入，不负责改变 tau、epsilon 或数学损失。ZeroValidPixelsError 继承 ValueError：调用者可精确捕获无监督异常，但正式 runner 应停止，不能 catch 后静默继续。Supervision 为 frozen dataclass，字段 valid/rainy/rainy\_valid/clean\_rate 都是张量，n\_valid/n\_rain 是 Python 整数；frozen 防止字段重新赋值，不会把内部 Tensor 变成真正只读。

image\_tensor(name,value,channels) 检查 torch.Tensor、torch.strided、四维 [B,C,H,W] 及全部维度 >0。channels 只接受调用方固定 1 或 32。strided 是普通 dense 张量布局，区别于 sparse；不要求 contiguous，因此合法非连续切片仍可运算。该函数不检查浮点 dtype，具体入口随后检查，职责不要混淆。

same\_shape\_device(name,value,reference) 要求精确同 shape 和 device，返回 None 代表通过，失败抛 ValueError。boolean\_mask 再要求 bool/dense。mask 不能用 float0/1 替代，也不能把 [H,W] 悄悄扩成 [B,1,H,W]；未来调用者若需构造 batch 掩膜必须明确产生合同形张量。

finite\_tensor(name,value) 调用 torch.isfinite(value).all() 再转 Python bool；CUDA 会同步。NaN 和 ±Inf 都拒绝。occurrence\_logits 要求实浮点 FP16/BF16/32/64、CPU 或 CUDA、单通道及全域 finite；quantiles 要求 32 通道 CPU/CUDA，并复用原 validate\_log\_quantiles 的 FP64/支撑/严格序，不自己发明 epsilon 或 clamp。

graph\_zero(value,dtype=None) 的 reshape(-1) 把元素拉平，[:0] 取零元素切片，sum(dtype=...) 产生零维 0。它不先把所有值加起来，所以即使总和会溢出也能生成连接原图的 0；梯度是全零。这里的零不是把非法输入修好，前置守卫仍先拒绝 NaN/Inf。

supervision(logit,qlog,rate,imerg\_valid,yunnan\_mask) 先检查两头与空间/device，再检查原 float32 非梯度 rate 和独立 bool masks，构造交集并计数。计数 0 停止；有效参考负/非有限停止；invalid 参考临时 where 成 0。rainy 在原精度与 torch.tensor(.1,float32) 比较，再与 valid 相交。返回的 clean\_rate 稍后才 double()/log1p。

具体边界：float32(.1) 提升后数值略高于十进制.1，所以先 double 再比较会错误地把边界值标有雨；测试 B1 保存三个 nextafter 相邻值。invalid 参考 NaN 允许进入临时 clean 副本，但原值不改；invalid 模型 qlog 的 NaN 仍拒绝，因为模型输出守卫覆盖全域。零 batch、32→31 通道、非 boolmask、meta/CPU 设备不一致都不靠广播或 fallback 继续。

本模块的 shape/dtype/finite/异常类属于工程保护；监督交集、严格 float32 雨标签、q 支撑/单调是冻结科学语义。未来正式网格 [100,100]、3430 格点及样本资格必须由另外获授权的 runner 检查，本 loss 保护不单独证明地理配准正确。

## 4 Focal 候选实现逐段解读

阅读 config.py、validation.py 后，再打开 focal.py。occurrence\_numerator(logit,rainy,valid,\*,alpha,gamma) 返回零维 S\_occ 张量，不除以 batch、有效像元数或有雨数。星号 \* 表示 alpha/gamma 只能用关键字传递，减少把两参数顺序写反的风险。类型注解用于阅读；真正拒绝错误输入的是函数中的运行时检查。

logit=[B,1,H,W] 是未 sigmoid 的实数分数；rainy/valid 必须为同形、同设备的 bool。rainy 来自 rate 原 float32 严格大于 float32(.1)；Focal 裸函数不自行根据概率产生标签。valid 是 IMERG 有效性与云南掩膜交集，与 rainy 是两个不同概念。

设  $b = \text{BCEWithLogits}(l, z)$ ,  $pt = \exp(-b)$ ,  $\alpha\_t = \alpha * z + (1 - \alpha) * (1 - z)$ , 则  $S\_occ = \sum\_valid \alpha\_t * (1 - pt)^\gamma * b$ 。BCE 用 reduction='none' 保留逐元素值，再执行 focusing 和求和；不能先求 mean 再拿 scalar 计算 pt。稳定 BCE 直接吃 logits，不能先 sigmoid 再传给 BCEWithLogits。PyTorch 2.7 BCE API 给出了输入与 reduction 的定义；项目真实数学来自 src/yuntapr/losses/focal.py。

第一个代码段调用 finite\_number: Python 中 bool 是 int 的子类，所以用 type(value) 精确排除 True/False; NaN 与 Inf 也拒绝。裸算术允许 alpha 属于 [0,1]、gamma>=0；正式候选配置只允许三个预声明组合，不开放参数搜索。随后检查 logit 与两个 masks，拒绝隐式广播。

第二段选择运算 dtype。logit.float() 的等价.to(torch.float32) 转换不是 detach：例如原 BF16 叶子仍连接 BCE 的计算图，backward 后 grad 保留原叶子的 dtype。with torch.autocast(...,enabled=False) 只在本块内固定运算精度，退出后恢复外层上下文。

logit[valid] 是 bool 索引，将有效位置收集为 [N\_valid]；rainy[valid] 同顺序收集标签。这样 invalid 像元不进入分子，避免先做乘 mask 再指望 NaN\*0 消失。这里仍先检查全域 logit finite，不能用索引隐藏坏输出。无 valid 的裸函数返回 graph\_zero；总损失入口则提前拒绝零监督。

torch.full\_like(logits,alpha) 产生同 dtype/device 的类别权重；torch.where 根据每个 hard label 选择 alpha 或 1-alpha。alpha=.5 时两类权重都为 .5，正负类别不能在 E1 改为权重 1。gamma==0 使用  $\alpha\_t * b$ ，避开不必要的 focusing 运算并保留系数；gamma2 使用 exp、pow 和 sum，与原 E0 定义一致。

F1/F3 的 l=0、gamma2 单像元期望  $\ln 2/8$ ；F2 的 gamma0 为  $\ln 2/2$ ；F4 两个相同有效元素为  $\ln 2$ 。103 项初轮套件包含这些手算检查；最终套件还覆盖不平衡 100 干/1 湿、±1000 和 ±1e30 logits、FP32/64 及低精度输入。所有数值属于合成教学，不是科研评价结果。

gamma0 的单元梯度为  $\alpha\_t * (\text{sigmoid}(l) - z)$ 。gamma2 还需要对 focusing 权重求导，不能把 pt.detach()；测试用完整解析式和 FP64 gradcheck/gradgradcheck 对照。valid=false 的位置梯度为 0。finite 检查是工程保护；alpha/gamma、BCE 及 focusing 是科学定义。

小练习：独立手算 l=0,z=false,gamma0 的值和梯度；解释为什么两类 l=0 损失相同而梯度符号相反；观察 sum 与 mean 的二倍关系；找出哪些行若删掉 .5 会改变 E1。学习练习不要求重新实现才能开展本轮候选开发，但正式接入前研究者仍需理解和审查。

## 5 Conditional Pinball 逐段解读

pinball.py 的 frozen\_taus(device=None) 永远返回 FP64 的 32 个值:  $(i-.5)/32$ ,  $i=1..32$ , 最后为 .984375。torch.arange(1,33) 的右端不包含 33; 先减 .5 再除 32, 不是从 0 到 1 的等间距端点集合。device 参数仅决定 CPU/CUDA 位置, 不改变 tau 或科学定义。

conditional\_pinball\_numerator(qlog, log\_target, rainy\_valid) 是公开入口。

qlog=[B,32,H,W], log\_target=[B,1,H,W], rainy\_valid=[B,1,H,W] bool; 全为同设备, 浮点路径必须 FP64。qlog 单位是  $\log_{1p}(\text{mm/h})$ , 不是 mm/h。公开入口复用冻结 qlog finite、最低支撑和严格序检查, 不允许 sort、clamp、nan\_to\_num 或 expm1。

log\_target 必须有限非负; 被 rainy\_valid 选中的 log\_target 必须严格高于  $\log_{1p}(\text{float32}(.1))$  提升至 FP64)。该检查防止调用者错误地把干像元标成条件有雨。完整接口先在原 float32 构造雨标签, 再提升 clean\_rate.double() 做  $\log_{1p}$ ; 不允许先提升再与十进制 .1 比较。rainy\_valid=false 的高参考值可能只是被独立有效性或云南掩膜排除, 不要求反向等价。

error=log\_target-qlog 的广播仅沿明确的 32tau 轴; 不是允许 mask、batch 或空间维随意广播。 $e>0$  表示模型分位数低于参考, 惩罚  $\tau \cdot e$ ;  $e<0$  使用  $(\tau-1) \cdot e$ , 两者均非负。 $\rho = \max(\tau \cdot e, (\tau-1) \cdot e)$ , 每个有雨像元先 mean32, 再把像元损失 sum, 得到未加权  $S_{qr}$ 。

\_pinball\_from\_errors 是私有内核。error.movedim(1,-1) 把 [B,32,H,W] 变为 [B,H,W,32]; rainy\_valid.squeeze(1) 去掉单例通道, 然后 bool 索引取出 [N\_rain,32]。tau=[32] 的乘法对应最后一轴; mean(dim=-1) 只平均 tau, 不平均雨像元。最后的.sum() 不含任何 N\_valid/N\_rain 分母。

P1-P4 的常数  $e=+1/-1/0$  和空 mask 只测试内核算术: 前两者  $\text{mean}(\tau)=\text{mean}(1-\tau)=.5$ , 第三 0, 第四返回连接计算图的 0。这些常数 signed errors 不是常数 qlog。常数 qlog 会在公开入口被严格序守卫拒绝, 不通过“关闭守卫”来运行手算测试。

无雨返回 error.reshape(-1)[:0].sum()。[:0] 是空切片, sum 得到 0 且仍连接输入图; backward 产生实际零梯度, 而不是 None。与  $\text{qlog.sum()} \cdot 0$  的数学零相同, 但避免很大有限 qlog 先求和溢出后形成  $\text{Inf} \cdot 0$ 。该路径属于工程实现保护, 没有新增科学损失。

对  $e>0$ ,  $dS_{qr}/d\tau_i = -\tau_i/32$ ;  $e<0$  则为  $(1-\tau_i)/32$ 。 $e=0$  是折点, 不能用中央差分要求唯一经典导数。torch.maximum 给出可用的子梯度, 但确认性梯度测试刻意远离折点。[PyTorch autograd 说明](#) 可辅助理解反向图; 本项目分段梯度来自实际 Pinball 定义。

本轮测试包括全部 32tau 解析梯度、合法严格序 q、非有限输出、FP32q 拒绝、干目标误选、shape/device 错误、超出物理 FP64 expm1 边界的有限 qlog。后者只证明 log 域算术没有偷偷 materialize 物理值; 不证明巨大条件雨强物理可信。

小练习: 对  $\tau=.984375$  分别手算  $e=1/-1$  的惩罚; 解释为何高  $\tau$  更惩罚低估; 将 mean32 误改 sum32 会使哪个量变 32 倍; 解释覆盖率可因过宽分布提高, 却不能单独证明质量改善。

## 6 总损失、监督张量与指标分母

`candidate_loss(logit,qlog,rate,imerg_valid,yunnan_mask,*,config)` 是候选聚合入口;config 必须为闭合 Ablation-Config, 不能用未验证 dict 或临时 gamma/lambda 代替实验 ID。再次 `__post_init__` 校验防止外部代码绕过 frozen 赋值保护后传入不匹配参数。该验证不是人类审批。

第一段 supervision: 输出两头先通过形状、设备及 finite/qlog 守卫; 参考 rate 必须原 float32、不参与 autograd; 两个 mask 必须独立 bool 且精确同形。 `valid=imerg_valid & yunnan_mask`, `N_valid=int(valid.sum().item())`, `rainy=clean_rate>float32(.1)`, `N_rain=sum(rainy&valid).item()` 把零维张量转为 Python 计数, 也会在 CUDA 同步, 属于安全检查成本而非正式性能测量。

`clean_rate=torch.where(valid,rate,zeros_like(rate))` 仅创建计算副本。有效参考 NaN/Inf/负值拒绝; invalid 参考允许 NaN 并临时置 0, 与原总损失一致, 测试检查输入未修改。模型输出全域的 NaN/Inf 都拒绝, 不能因 invalidmask 而修补。 `N_valid=0` 抛 `ZeroValidPixelsError`; 正常 batch 不静默 skip, 也不假称发生了更新。

第二段调用 Focal 得到 `S_occ`, 再将 `clean_rate` 提升 FP64 并 `log1p`, 调用 Pinball 得到 `S_qr`。发生分子已包含 alpha/gamma, 但 “unweighted” 表示没有 `lambda_q`; 条件分子始终无 `lambda`。第三段 `combine_numerators(S_occ,S_qr,N_valid,*,lambda_q)` 要求有限非负 FP32/64 标量、同设备、正整数 `N_valid` 及正有限 `lambda`, 计算  $(S_{occ} + \lambda S_{qr}) / N_{valid}$ 。

示例 T1: `S_occ=2, S_qr=3, N_valid=10`, `lambda1` 得到 .5, `lambda2` 得到 .8。不是把所有 loss 乘 2, 也不是两处都乘 `lambda` 导致平方权重。T2 科学 CPB= $3/2=1.5$ ; `N_rain` 只用于科学条件评价, 不用于训练分母。这些都是纯合成手算与单元测试值。

`CandidateLossResult` 保留 `experiment_id`, `s_occ`, `s_qr`, `training_objective`, `weighted_quantile_per_valid`, `n_valid`, `n_rain`, `conditional_skipped`, `status` 及 `execution_authorization=false`。科学 CPB 属性只返回 `s_qr/n_rain`; 无雨返回 None, 表示不可估计, 不是模型完美。`weighted_quantile_per_valid` 是辅助训练分解, 不可拿它冒充 unweighted CPB。

`metrics.py` 的 `ReportingSums.from_candidate` 用 detach 再 float 生成只读汇总, 停止构建梯度图但不修改训练张量。`conditional_pinball` 按有雨数除; `pooled(other)` 累加分子和计数, 再计算比值, 不能把不等 batch 的 CPB 直接平均。`common_validation_sums` 在 `no_grad` 上下文显式使用 FP64 logits 及 E0, 固定 `gamma2/lambda1`; 训练臂的 gamma 或 lambda 不会改变共同 core 口径。

发生梯度和分位数梯度都通过 total 标量相连。gamma0 时 logit 梯度 =  $.5 \cdot (p-z) / N_{valid}$ ; q 梯度为分段 Pinball 梯度再乘 `lambda/N_valid`。E2 的 q 梯度为 E0 的 2 倍, 而同一输入的发生梯度保持不变。这个合成张量性质不保证共享骨干真实训练后的发生输出不变。

当前 `candidate_loss` 可以被 Python 调用, 但没有数据读取或执行授权能力; 真正 runner 必须在外层验证科学审批、身份、资源及执行范围。学习时只使用测试文件里的小型合成输入。本轮不生成新的真实 Brier、AUROC、AP、Pinball 或上尾验证结果。

## 7 未加权评价接口解读

metrics.py 没有 Brier/AUROC/AP 实现、没有模型推理，也没有读取样本。它只把调用者已经提供的分子与计数按冻结口径整理。正式数据是否允许传入由未来执行 gate 控制；本轮所有调用都是合成张量。

ReportingSums(s\_occ,s\_qr,n\_valid,n\_rain) 是 frozen dataclass。\_\_post\_init\_\_ 检查两个分子为真正 Python 数且 finite/非负，计数是真正 int、 $N\_valid > 0$ 、 $0 \leq N\_rain \leq N\_valid$ ；bool 不接受。对象保存的是 unweighted sums，不能放  $\lambda * S\_qr$  进去冒充原  $S\_qr$ 。接口语义由 from\_candidate 与测试确保，任意外部人手输入 float 则仍需来源审查。

from\_candidate(result) 是 @classmethod，首参 cls 代表当前类，不是对象 self；它用  $\text{float}(\text{result.s\_occ.detach}())$  和 s\_qr 生成汇总。detach 不改变数值，而是切断本报告分支的梯度图。GPU 到 Python scalar 会同步，因此不能把汇总接口时间当纯训练吞吐。

conditional\_pinball 是 @property，使用者写 report.conditional\_pinball 而不是函数括号；返回  $S\_qr/N\_rain$ ， $N\_rain=0$  返回 None。pooled(other) 先加分子及计数，再创建经过同样检查的新对象，避免平均两个 batch 比例。它不合并 seed 模型预测，也不把 seed $\times$  像元当独立重复。

common\_validation\_sums(logit,qlog,rate,imerge\_valid,yunnan\_mask) 被 @torch.no\_grad() 修饰，这些运算不建训练梯度图。它将 logit.double()，显式 get\_config('E0') 后调用同一受保护算术入口，得到  $\gamma_2/\lambda_1$  的 FP64 common core 分子。训练 E1 的  $\gamma_0$  或 E2 的  $\lambda_2$  绝不能从训练 result 直接替换共同 core。

例 T2:  $S\_qr_3$ 、 $N\_rain_2$ ，CPB1.5；把这个对象与自己 pooled 后  $6/4$  仍 1.5。若两个 batch 雨数不等，则应使用总  $S\_qr$ /总  $N\_rain$ ，不是两个 CPB 简单平均。所有示例均为合成教学数值。未来指标还须绑定年份、域、样本身份、missingness 及原科学审查规则，当前接口不是完整科学 observer。

## 8 E0 冻结数学兼容性：实际合成验证

证据: tests/synthetic\_attempt\_008.json 与 XML、tests/phase\_b\_v2\_ablation\_implementation/test\_losses.py 及 test\_gradients.py。最终项目环境 CPU177/0、CUDA3/0；数字是检查数量，不是模型评价样本或天气事件。没有真实数据或模型推理。

| 核验                 | 实际范围                                     | 预先固定判据                                         | 结果   |
|--------------------|------------------------------------------|------------------------------------------------|------|
| Focal 手算 F1-F4     | 单/双元素，正负标签，<br>gamma0/2                  | FP64 relle-12、abs1e-14                         | PASS |
| 原 focal 函数         | FP32/FP64/BF16/FP16 输<br>入，gamma0/2      | FP32 路径 rtol8e-7、<br>atol1e-8；FP64 1e-12/1e-14 | PASS |
| 冻结 autocast 语义     | CPU BF16 logits, BCE<br>FP32, gamma0/2   | 8e-7/1e-8                                      | PASS |
| 原 total 函数与两头梯度    | B=1/2/3/5, H2/W3, 独立<br>invalid 与云南 mask | 8e-7/1e-8, 不调宽容差                               | PASS |
| 未加权原 Pinball       | 合法严格序 FP64q, 雨 valid<br>交集               | 1e-12/1e-14                                    | PASS |
| 共同 validation core | 原 FP64 E0 分子/共同分母                        | relle-12                                       | PASS |
| GPU loss 与两头梯度     | 三臂，同一小型 FP64 合成<br>输入                    | CPU/CUDA 1e-10/1e-12                           | PASS |
| 标签边界               | float32(.1) 及两侧 nextafter                | 只上邻有雨                                          | PASS |

相容表示合法受测输入下的数学值和梯度达到这些容差，不声明所有硬件、输入或逐位 bitwise 一致。候选先收集 valid 再 sum，原函数乘 mask 后 sum，归约顺序可能造成正常舍入差异。裸 FP16/BF16 在没有 autocast 时，原函数可能保持低精度；候选显式 FP32 BCE，比较的是冻结正式 autocast 预期算术，而不是宣称裸低精度逐位一致。

原 b0\_core\_loss 没有 lambda 参数，E0 只对应 gamma2/lambda1。E1 唯一改变 gamma，E2 唯一改变 lambda；同输入科学 CPB 在三臂完全相同，因为 q 与标签不变、S\_qr 无权重。未来真实训练改变共享参数后，科学结果仍未知。

有意加强的工程拒绝行为：参数 NaN/Inf/bool、mask/dtype/device/shape 错误、非单调或失去支撑的 qlog 明确拒绝；零有效监督提前抛错，旧 loss 先返回 batch\_skipped 再被正式 runner 拒绝。无雨仍连接 q 的零梯度；用空切片 sum 避免 sum\*q0 的中间溢出。invalid 参考沿用临时 clean 副本；invalid 输出仍严格 finite。

引用原文件字节 SHA 与重要函数行号见 source\_identity.json；原 loss、quantile 参数化、正式 Phase-A 入口和冻结合同未修改。E0 工程兼容不等于 Phase-A 科学验收，不追认 B1 历史恢复，也不能证明 Brier、q32 欠覆盖或极端上尾已解决。

最终边界复核的 CPU007 为 176 通过/1 失败：标量设备错配应在 finite 算术前拒绝。代码将 device 检查前移并保留同形 dtype 合同，容差未改；CPU008=177 通过/0 失败，CUDA004=3 通过/0 失败。详见 tests/scalar\_device\_repair.json 与原失败日志。

## 9 合成梯度与环境验证报告

最终项目运行时：Python3.12.14、PyTorch2.11.0+cu128、pytest9.1.1, RTX5060Laptop、CUDA12.8, 构建包含 sm\_120。来源 tests/project\_cuda\_environment.json。测试依赖安装在本任务.local/test\_dependencies, 未修改项目 CUDA 运行时依赖。没有测量正式训练 GPU 耗时、模型吞吐或显存预算。

CPU suite attempt008: 177 passed、0 failures、0 errors、0 skipped; CUDA attempt004: 3 passed、0 failures、0 errors、0 skipped。11 项静态检查通过。当前独立测试用例 180, 历史 attempt 不相加。CPU 用的是同一项目 CUDA Python 解释器但输入留在 CPU; CUDA 另开限时 60 秒进程。

13 个 CPU 梯度测试: gamma0/2 解析 Focal 梯度; gamma0/2 各一次 gradcheck 与 gradgradcheck; 全部 32tau 的正/负分段解析梯度; E0/E1/E2 的总损失双头 gradcheck; 三臂 q 梯度权重及单坐标中央差分; invalid 像元零梯度。计数按 pytest 实例为 13, gradgradcheck 等多个断言仍属于其所在一个实例。

gradcheck 固定 FP64、eps1e-6、atol1e-6、rtol1e-5; 单坐标差分 eps1e-6、rel2e-8、abs1e-9。测试点远离 Pinball 折点和 q 支撑/相邻序边界。不是通过增加容差或给 q 排序来制造通过。Focal 解析导数是  $\alpha_t \cdot (p-z)^{[m^{\gamma} + \gamma \cdot b \cdot p^t \cdot m^{\gamma-1}]}$ ,  $m=1-p$ ; gamma0 单独为  $\alpha_t \cdot (p-z)$ 。Pinball 对 q 的导数在欠预测处为  $-\tau/32$ , 在过预测处为  $(1-\tau)/32$ , 再乘  $\lambda/N_{\text{valid}}$ 。

无雨检查中 q.grad 是实际零张量, 不是 None; 有效发生梯度保留。无效像元两头梯度 0。P3 折点仅检查值、finite 与可反传, 不断言唯一数学梯度。backward 只求合成叶子的梯度; 测试没有 Optimizer 实例或 step, 禁止 torch.load/save 和 expm1 的自动 fixture 会在误调用时抛错。

保留的实际尝试: attempt001 的 103 项初轮通过; CPU002 因导入冻结协议工具而缺 yaml, 出现 1 个 collection error; 改为只 AST 提取原 lr\_for\_update 纯函数, 不 import 其模块。CPU003=162、004=174 通过。旧 torch2.7.1+cu118 最小 CUDA 张量探测虽成功, 但完整 CUDA001 在 60 秒超时, 未形成测试完成数。找到历史工程记录的 cu128 环境后, CPU005/CUDA002 因缺 pytest 启动失败; pytest 安装到任务私有 target 后 CPU006/CUDA003 完成。相关完整输出均保留, 不把未启动/超时记为通过。

这些结果仅证明受测候选数学与保护逻辑。没有真实 2023/2024 样本、封存 2025、私有 checkpoint、新模型 forward 或正式验证推理。核心科学假设、效应界限、多重比较和资源执行授权仍未批准。

最终边界复核的 CPU007 为 176 通过/1 失败: 标量设备错配应在 finite 算术前拒绝。代码将 device 检查前移并保留同形 dtype 合同, 容差未改; CPU008=177 通过/0 失败, CUDA004=3 通过/0 失败。详见 tests/scalar\_device\_repair.json 与原失败日志。

## 10 独立 v2 runner 的非执行型集成提案

当前只有 `readiness.BlockedRunner`: `describe()` 返回 18 身份、预算、阻塞事项, `run(*args,**kwargs)` 无条件抛 `FormalExecutionBlocked`。传入 `approved=true` 或字符串 `AUTHORIZED_TO_EXECUTE` 也不能运行。没有训练 `loop`、`DataLoader`、模型实例、`Optimizer`、`checkpoint` 应用或旧 `FinalFit` 引用。

未来建议新增隔离 v2 runner, 而非修改 Phase-A 原入口; 因此本轮没有需要应用到正式路径的 diff 补丁。建议模块责任: 审定协议/执行授权读取与独立真实性验证; 数据防火墙与 2023/24 配对资格; 可复核 `fresh initializer`; `seed` 和固定样本排列; 训练 `objective` 适配本候选; 固定 E0 FP64 科学汇总; 逐 epoch 验证与 `immutable LAST` 事务; 异常停止与具体 `LAST` 独立恢复; 日志和资源审计。这里不实现实际训练启动。

Fresh 一致性接口 `readiness.py` 逐项解释: `TensorIdentity(shape, dtype, sha256)` 要求正整数 tuple、float32、64 位小写十六进制 SHA。`state_metadata_digest(tensors)` 将按参数名排序的声明 `serialize` 为 JSON 再 `hash`, 是声明层 `digest`, 不能替代真实 `tensor bytes` 哈希。`FreshRecord` 包含 `RunSpec`、张量映射、`state/RNG/order/initializer_code SHA` 及 `historical_checkpoint_loaded=false`; `validate` 重验字段与 `digest`。

`audit_fresh_metadata(records)` 要求完整 18 项、不重复; 同模型/`seed` 跨三臂 `full state digest` 一致; 同 `seed` 六 `run` 的 `post_pair RNG` 与样本 `order SHA` 一致; `initializer` 代码身份一致; B0/B1 名字集合相同, 只允许 `backbone.enc0.conv1.weight` 和 `backbone.enc0.skip.weight` 的 `shape` 不同, 其他声明完全相同。函数只检查输入 `metadata`, 不打开模型、权重或原始样本, 所以 `actual_initialization_proven=false`, 即使 `metadata_consistent=true`。

后续实际 `initializer` 仍要证明参数总数、同名同形拷贝、B1 两个不同形状权重保留 `native` 初始化及 `zero skip`、独立 `reseed`、没有 `historical optimizer/RNG` 转移、完整初态真实 SHA。正式 `shape` 必须冻结 501 输入/100 输出与模型参数量; 本轮 `tiny metadata fixture` 不证明真实模型满足这些条件, 也不构造这 18 个模型。

`review_checkpoint_binding(metadata, expected, resume_approval_reference=None)` 要求封闭字段: `run/LAST/model/optimizer/RNG/scheduler/code/protocol/data/init SHA`、完成 `epoch`、`retained updates`、完成 `train+validation` 边界、原执行事件 `reference`。`epoch1..9`, `updates=epoch*5228`, `expected` 明确绑定六项 `run/LAST/code/protocol/data/init`。`epoch9 remaining=0`, 不能继续训练。函数不加载或应用状态; 即使给 `reference` 字符串, `approval_authenticity_verified=false`、`can_apply_checkpoint_state=false`。

这些字符串只是合成 `fixture` 或待独立核验的来源引用, 不是生成审批。未来恢复须真实研究者独立决定并绑定具体 `LAST SHA`、原运行祖先、剩余预算、额外重放成本, 再由经审查的正式 `gate` 处理; 每次独立批准, 禁止自动 `retry/resume`。原 B1 `LAST14` 恢复偏差继续 `NOT_GRANTEDED`。

训练和评价必须分离: `candidate_loss` 按各臂 `gamma/lambda` 优化; `metrics.common_validation_sums` 固定 E0 FP64 共同 `core`, `CPB` 始终 `unweighted S_qr/N_rain`。18`run` 固定 `epoch9`, 不使用新成绩选择 `BEST` 或改阈值/样本。阶段 1 `seed2026` 完整六 `run` 和阶段 2 另两 `seed` 十二 `run` 各有独立执行批准; 阶段 1 成绩不能修改已批准规则或自动启动阶段 2。

安全接续顺序: 核对本版 `Git/SHA` 与失败记录 → 研究者学习并审查代码、处理科学待决字段 → 独立批准候选数学及正式集成范围 → 另版工程化新 `runner` 与获授权的模型/数据 `preflight` → 绑定真实初始化、数据、资源身份 → 另行批准具体阶段执行。当前开发委托已覆盖候选 `loss` 合成 `forward/backward`, 但不覆盖这些后续正式步骤。

## 11 元数据审核与阻断 runner 逐段解读

readiness.py 不 import torch、模型、数据、optimizer，也不打开文件。其参数是 Python dataclass/dict/Sequence；通过包公开入口导入仍会先执行包 `__init__` 并加载 loss 依赖，不能将整个包称为零 torch 依赖。static\_review.py 才是标准库独立入口。

`require_sha(value,name)` 用 `re.fullmatch` 要求完整 64 位小写十六进制字符串，而非只检查某个子串。它验证格式，不证明 SHA 对应真实文件、更不证明人类授权。`TensorIdentity(shape,dtype,sha256)` 的 `shape` 必须 tuple 且每维正整数，`dtype` 精确 float32；`__post_init__` 在构造时运行。

`state_metadata_digest(tensors)` 要求非空 dict，key 是非空参数名，value 是 `TensorIdentity`；sorted 按名字稳定排序，asdict 递归把 dataclass 转 dict，`json.dumps(sort_keys=True,separators=...)` 固定表示，然后 `hashlib.sha256(body.encode()).hexdigest()` 生成 64 位 digest。该 digest 是声明 JSON 的身份，不能等同真实 tensor bytes hash 或加载后的全 model SHA。

FreshRecord 字段及 validate 见集成提案；validate 再次检查 RunSpec、四个 SHA、historical\_checkpoint\_loaded 严格 False 以及声明 digest。audit\_fresh\_metadata 用 run\_id 建查找 dict，再核对完整 18 集合，避免重复项被 dict 静默覆盖。不同 seed 的 hash 没有自动相等约束；同 seed 跨臂和跨模型按照冻结匹配规则检查，共享 shape 权重须一致。

集合运算 `different={name:shape 不同的名字}` 在实际代码用 set 推导式；必须恰好两个冻结输入权重名，不允许新增第三个架构差异。其他权重比较的是 `TensorIdentity` 全字段而非只 shape。这里未检查真实参数量、native skip zero 或 RNG 来源，因为没有实际模型；输出明确 `actual_initialization_proven=false`。

`review_checkpoint_binding(metadata,expected,*,resume_approval_reference=None)` 只检查封闭字段、SHA 格式、完成 epoch 预算与原事件 reference、具体六项 expected 绑定。expected 必须完整，不接受遗漏字段。即使 reference 有字符串也只是“存在待核验引用”，不会设置 `approval_authenticity_verified=true` 或 `can_apply_checkpoint_state=true`。epoch9 无剩余更新，next\_epoch 为 None。

FormalExecutionBlocked 继承 RuntimeError。BlockedRunner.describe 返回设计信息；`run(*args,**kwargs)->NoReturn` 可以接任意伪 flag，但函数唯一行为是 raise。NoReturn 是类型提示，不是安全机制；真正阻断来自无条件 raise 和没有执行分支。测试传入 False/True/APPROVED/AUTHORIZED\_TO\_EXECUTE 四种值都确认抛错。

练习：在合成 metadata 副本中改一个共享 tensor SHA，预测哪个比较拒绝；删一个 run 或重复一项，解释为何集合检查仍能发现；把 completed\_epoch 改 9，手算 remaining=0；解释 hash 一致、metadata 一致、真实初始化解证据与独立批准为什么是四件不同事。

## 12 研究者代码与科学审查清单（未批准）

这是待审事项，不是审批记录；没有签名、批准 token 或已批准勾选。当前研究者已委托候选开发与合成验证，以下真实独立审查仍需后续发生。

- 阅读闭合 config，确认 E0/E1/E2 科学假设与唯一因素；代码 PASS 不能替代效应界限、副作用容忍、多重比较和功效判断。
- 解释 Focal 为何用 logit 及 BCEWithLogits；gamma0 为何仍是.5BCE；理解 focusing 梯度不是只对 BCE 求导。
- 解释 FP32 雨标签提升前比较、独立 bool masks、N\_valid/N\_rain 两个分母、invalid 参考临时清理与输出 NaN 拒绝。
- 解释 32tau、FP64 log 域、mean32 后 sum、q 支撑与严格序；区分私有 signed-error 手算内核和公开 qlog 接口。
- 核对 total 仅乘一次 lambda、unweighted 分子保留、无雨 q 梯度连接、零有效明确停止；接受或提出独立候选修订，不能直接改冻结定义。
- 阅读 E0 对原函数及两头梯度的实际兼容证据、固定容差、gradcheck 远离折点的原因；理解合成 PASS 不证明真实模型或物理性能。
- 阅读全部失败 attempt，尤其旧 CUDA timeout、缺 yaml/pytest 及任务私有依赖路径；正式 GPU 资源与正式依赖锁定仍未测量或批准。
- 理解 metadata fresh/恢复检查不能证明真实权重来源，也不能验证人类批准真实性；BlockedRunner 无启动能力。
- 独立决定 Phase-A 接受范围与 B1 历史恢复治理处置，保留 NOT\_GRANTED；本候选不自动追认。
- 独立批准正式集成范围后，在新版本工程化 runner，开展另获授权的真实数据/模型 preflight；再批准阶段 1/2 具体执行 scope、SHA、资源和恢复规则。

现有 2024 开发验证已经用于 BEST 与反复诊断，不是全新确认集；q32 欠覆盖、上尾异常与 CPB 改善未稳健确认的限制继续保留。2025、FinalFit、新架构、校准器均不纳入本候选开发的正式执行范围。

## 13 研究者代码学习手册与注释源码副本

本轮研究者已委托 Codex 先实现候选，再由研究者学习与独立审查。以下副本与实际候选源码按 SHA 绑定，不是另一份可导入模块；修改学习副本不会修改真实实现。所有代码属于候选，最终科学决定仍由研究者作出。

### 13.1 建议学习顺序与小练习

- `config.py`: 先理解函数/关键字参数、`dataclass`、`tuple` 和 `type` 检查。练习写出三臂的唯一不同项，手算 18 身份和每 `run47052` 更新。
- `validation.py`: 理解 `[B,C,H,W]`、`bool` 索引、`shape/device/dtype` 与交集。练习画出两个 `mask` 交集，并解释 `invalid reference` 临时 `clean` 不等于修改源数据。
- `focal.py`: 对照 F1-F4，先手算 `gamma0` 值/梯度，再理解 `gamma2` 对权重也求导。练习指出误删 `alpha` 会如何改变 `E1`。
- `pinball.py`: 先看 `signed errors` 内核，再看公开严格 `q` 入口。练习手算高 `tau` 欠/过预测惩罚，解释 `mean32` 与 `sum` 像元的顺序。
- `total.py` 和 `metrics.py`: 用 `T1/T2` 核对两个分母、`lambda` 一次与无雨 `None`。练习合并两个不同雨数 `batch` 的 `CPB`。
- `test_gradients.py`: 认识 `requires_grad`、叶子、`grad_fn`、`backward`、`autograd.grad` 及 `finite difference`。练习从链式法则推导 `q` 梯度的 `lambda/N_valid` 因子。
- `readiness.py`: 阅读 `metadata` 而非真实 `state`，尝试仅在个人合成副本中更改一个 `SHA`，理解 `BLOCKED` 与数学 `PASS` 的区别。

### 13.2 共同语法与计算图

`from .config` 是包内相对导入，`__all__` 只定义公开名称，不授予权限。→`Tensor`、`tuple[int,...]` 等是类型提示，不能替代 `if/raise` 检查。`@dataclass` 自动生成构造与比较，`frozen` 阻止普通字段重赋值但不冻结内部 `dict/Tensor`。`@property` 允许以属性形式读派生值；`@classmethod` 的 `cls` 代表类。

Python 中的 `and/or` 短路有助在错误 `dtype` 之前停止；`type(x)is int` 排除 `bool`。`with` 上下文离开后恢复 `autocast` 设置。`torch.where` 返回新 `Tensor`；`bool` 索引选择元素，`.movedim` 重新安排轴，`.squeeze(1)` 只删单例通道。`reshape` 不等于 `detach`，类型转换也保留可微路径。

`forward` 是由输入算出 `loss` 并构图；`backward` 按链式法则累加叶子。`grad`。`autograd.grad` 直接返回梯度，默认不会像 `backward` 一样累加到 `grad`。`detach/no_grad` 用于报告分支，不能放到训练 `loss` 中截断梯度。`gradcheck` 把自动微分与 `FP64` 中央差分比较；在折点不适用唯一经典导数。当前仅合成叶子参与图，没有模型参数更新或 `Optimizer`。

工程保护包括 `finite/shape/device/type` 拒绝、`SHA` 检查和 `BlockedRunner`；科学定义包括 `alpha/gamma`、`32tau`、`log` 域、雨阈值和分母。两者都重要，但工程 `PASS` 不能自动批准科学假设。[PyTorch autograd](#) 及 [混合精度说明](#) 提供 API 背景；项目公式与来源定位见对应 `walkthrough/source_identity`。

### 13.3 测试和交付工具也应理解

`conftest.py` 的 `autouse fixture` 在每个测试前禁止 `step`、`torch.load/save` 和 `expm1`，结束后由 `monkeypatch` 恢复原方法。`synthetic_case` 只用 `linspace/arange/tensor` 构造小张量，`clone().requires_grad_()` 建立可观察梯度的叶子。`rate/masks` 没有梯度。`test_losses` 用手算和冻结函数，不与新实现自己互证；`test_gradients` 用解析式/差分；`test_readiness` 只改内存 `metadata`；`test_cuda` 是可选独立进程的三臂 `CPU/GPU` 对照。

run\_checks.py 调用 pytest 子进程而不是模型 runner, 60 秒 timeout 会终止自己启动的测试进程, 保存完整输出与 XML, 不中断别人的任务。static\_review.py 只 AST 与 SHA, 无 torch import; author\_delivery.py/后续 export/close 工具处理文档和元数据, 不读观测或权重。若修改真实候选源码, 必须重新测试并用新版本身份归档, 不能只改副本或覆盖已发表结果。

### 13.4 \_\_init\_\_.py: 实际源码、函数位置与副本

来源 src/yuntapr/experimental/phase\_b\_v2\_ablations/\_\_init\_\_.py。

SHA256: 55207122a41dd3112f3933750f78a1e2318249a38b01f37f4d00badec3de418b。参数语义见对应 walk-through; 下面是实际完整注释源码副本。

| 函数或方法 (参数名)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | 源码行 | 阅读入口 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|------|
| <pre> 1 """Candidate loss APIs only: no model, data loader, optimizer or training loop.""" 2 from .config import AblationConfig, RunSpec, get_config, proposed_runs 3 from .focal import occurrence_numerator 4 from .pinball import conditional_pinball_numerator 5 from .total import CandidateLossResult, candidate_loss, combine_numerators 6 7 __all__ = ["AblationConfig", "RunSpec", "get_config", "proposed_runs", 8           "occurrence_numerator", "conditional_pinball_numerator", 9           "CandidateLossResult", "candidate_loss", "combine_numerators"] </pre> |     |      |

### 13.5 config.py: 实际源码、函数位置与副本

来源 src/yuntapr/experimental/phase\_b\_v2\_ablations/config.py。

SHA256: 5ae850c28417b555dc7825bd204f95f8275bc3ddc5b64cbfa230a2517d4d5e3a。参数语义见对应 walk-through; 下面是实际完整注释源码副本。

| 函数或方法 (参数名)                  | 源码行 | 阅读入口                  |
|------------------------------|-----|-----------------------|
| finite_number(name, value)   | 12  | 完整 type/default 见下面源码 |
| get_config(experiment_id)    | 34  | 完整 type/default 见下面源码 |
| proposed_runs()              | 62  | 完整 type/default 见下面源码 |
| workload()                   | 66  | 完整 type/default 见下面源码 |
| learning_rate_prefix(update) | 75  | 完整 type/default 见下面源码 |
| __post_init__(self)          | 26  | 完整 type/default 见下面源码 |
| __post_init__(self)          | 46  | 完整 type/default 见下面源码 |
| run_id(self)                 | 54  | 完整 type/default 见下面源码 |
| stage(self)                  | 58  | 完整 type/default 见下面源码 |

|                                                                                                                                                                                                                                                                                                                                                                                                      |  |  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
| <pre> 1 """Three closed, unapproved loss conditions and 18 candidate run identities.""" 2 from __future__ import annotations 3 from dataclasses import dataclass 4 import math 5 6 _PARAMETERS = {"E0": (.5, 2., 1.), "E1": (.5, 0., 1.), "E2": (.5, 2., 2.)} 7 MODELS = ("BO_MATCHED_V2", "B1_V2") 8 SEEDS = (2026, 2027, 2028) 9 PROTOCOL_STATUS = "PROPOSED_FOR_RESEARCHER_APPROVAL" 10 11 </pre> |  |  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|

```

12 def finite_number(name: str, value: object) -> float:
13     """Reject booleans, strings, NaN and infinity rather than silently coerce."""
14     if type(value) not in (int, float) or not math.isfinite(value):
15         raise ValueError(f"{name}: finite Python int/float required, excluding bool")
16     return float(value)
17
18
19 @dataclass(frozen=True)
20 class AblationConfig:
21     experiment_id: str
22     alpha: float
23     gamma: float
24     lambda_q: float
25
26     def __post_init__(self) -> None:
27         if type(self.experiment_id) is not str or self.experiment_id not in _PARAMETERS:
28             raise ValueError("Unknown experiment ID; only E0/E1/E2 are candidates")
29         values = tuple(finite_number(k, getattr(self, k)) for k in ("alpha", "gamma", "lambda_q"))
30         if values != _PARAMETERS[self.experiment_id]:
31             raise ValueError("Experiment parameters do not match the closed candidate condition")
32
33
34 def get_config(experiment_id: str) -> AblationConfig:
35     if type(experiment_id) is not str or experiment_id not in _PARAMETERS:
36         raise ValueError("Unknown experiment ID; no parameter search or automatic selection")
37     return AblationConfig(experiment_id, *_PARAMETERS[experiment_id])
38
39
40 @dataclass(frozen=True)
41 class RunSpec:
42     experiment_id: str
43     model: str
44     seed: int
45
46     def __post_init__(self) -> None:
47         get_config(self.experiment_id)
48         if type(self.model) is not str or self.model not in MODELS:
49             raise ValueError("Unknown candidate model identity")
50         if type(self.seed) is not int or self.seed not in SEEDS:
51             raise ValueError("Candidate seeds are exactly 2026/2027/2028")
52
53     @property
54     def run_id(self) -> str:
55         return f"{self.experiment_id}_{self.model}_s{self.seed}"
56
57     @property
58     def stage(self) -> int:
59         return 1 if self.seed == 2026 else 2
60
61
62 def proposed_runs() -> tuple[RunSpec, ...]:
63     return tuple(RunSpec(arm, model, seed) for seed in SEEDS for arm in _PARAMETERS for model in MODELS)
64
65
66 def workload() -> dict[str, int]:
67     """Pure arithmetic; these counts do not mean that any updates took place."""
68     train_batches, val_batches = (10455 + 1) // 2, (10501 + 7) // 8
69     return {"runs": 18, "epochs": 9, "updates_per_epoch": train_batches,
70           "updates_per_run": 9 * train_batches, "updates_total": 18 * 9 * train_batches,
71           "validation_batches_per_epoch": val_batches, "validation_batches_total": 18 * 9 * val_batches,
72           "stage1_updates": 6 * 9 * train_batches, "stage2_updates": 12 * 9 * train_batches}
73
74
75 def learning_rate_prefix(update: int) -> float:
76     """First nine epochs of the original 50-epoch trajectory, no optimizer."""
77     if type(update) is not int or not 1 <= update <= 47052:
78         raise ValueError("Candidate one-based update must be in [1,47052]")
79     if update <= 5228:
80         return 1e-4 * (update / 5228) # warmup is not clamped to the cosine floor
81     progress = (update - 5228) / (261400 - 5228)

```

```
82 return 1e-6 + (1e-4 - 1e-6) * (1 + math.cos(math.pi * progress)) / 2
```

### 13.6 validation.py: 实际源码、函数位置与副本

来源 src/yuntapr/experimental/phase\_b\_v2\_ablations/validation.py。

SHA256: 05885c309b6cf930a57f30b25438c8664be2ddc5a36eb9f86de82654e886beb0。参数语义见对应 walk-through；下面是实际完整注释源码副本。

| 函数或方法（参数名）                                               | 源码行 | 阅读入口                  |
|----------------------------------------------------------|-----|-----------------------|
| image_tensor(name, value, channels)                      | 12  | 完整 type/default 见下面源码 |
| same_shape_device(name, value, reference)                | 19  | 完整 type/default 见下面源码 |
| boolean_mask(name, value, reference)                     | 26  | 完整 type/default 见下面源码 |
| finite_tensor(name, value)                               | 32  | 完整 type/default 见下面源码 |
| occurrence_logits(logit)                                 | 37  | 完整 type/default 见下面源码 |
| quantiles(qlog)                                          | 46  | 完整 type/default 见下面源码 |
| graph_zero(value, *, dtype)                              | 54  | 完整 type/default 见下面源码 |
| supervision(logit, qlog, rate, imerg_valid, yunnan_mask) | 69  | 完整 type/default 见下面源码 |

```

1  """Explicit tensor contracts; no output repair and no physical conversion."""
2  from __future__ import annotations
3  from dataclasses import dataclass
4  import torch
5  from yuntapr.models.quantile_v2.outputs import validate_log_quantiles
6
7
8  class ZeroValidPixelsError(ValueError):
9      """Stop a loss call with zero supervision; never imply a skipped update."""
10
11
12  def image_tensor(name: str, value: torch.Tensor, channels: int) -> None:
13      if not isinstance(value, torch.Tensor) or value.layout != torch.strided:
14          raise ValueError(f"{name}: dense strided Tensor required")
15      if value.ndim != 4 or value.shape[1] != channels or any(n <= 0 for n in value.shape):
16          raise ValueError(f"{name}: nonempty [B,{channels},H,W] required")
17
18
19  def same_shape_device(name: str, value: torch.Tensor, reference: torch.Tensor) -> None:
20      if not isinstance(value, torch.Tensor) or value.shape != reference.shape:
21          raise ValueError(f"{name}: exact shape required; mask broadcasting is forbidden")
22      if value.device != reference.device:
23          raise ValueError(f"{name}: device mismatch")
24
25
26  def boolean_mask(name: str, value: torch.Tensor, reference: torch.Tensor) -> None:
27      same_shape_device(name, value, reference)
28      if value.dtype != torch.bool or value.layout != torch.strided:
29          raise ValueError(f"{name}: dense bool Tensor required")
30
31
32  def finite_tensor(name: str, value: torch.Tensor) -> None:
33      if not bool(torch.isfinite(value).all()):
34          raise FloatingPointError(f"{name}: NaN/Inf forbidden, including masked output pixels")
35
36
37  def occurrence_logits(logit: torch.Tensor) -> None:
38      image_tensor("logit", logit, 1)

```

```

39     if logit.dtype not in (torch.float16, torch.bfloat16, torch.float32, torch.float64):
40         raise ValueError("logit: real floating dtype required")
41     if logit.device.type not in ("cpu", "cuda"):
42         raise ValueError("logit: this candidate supports CPU/CUDA only")
43     finite_tensor("logit", logit)
44
45
46 def quantiles(qlog: torch.Tensor) -> None:
47     image_tensor("qlog", qlog, 32)
48     if qlog.device.type not in ("cpu", "cuda"):
49         raise ValueError("qlog: this candidate supports CPU/CUDA only")
50     # Reuse the frozen FP64, finite, support and strict-order guards unchanged.
51     validate_log_quantiles(qlog)
52
53
54 def graph_zero(value: torch.Tensor, *, dtype: torch.dtype | None = None) -> torch.Tensor:
55     """Empty-view sum gives connected zero without overflowing sum(value)*0."""
56     return value.reshape(-1)[:0].sum(dtype=dtype)
57
58
59 @dataclass(frozen=True)
60 class Supervision:
61     valid: torch.Tensor
62     rainy: torch.Tensor
63     rainy_valid: torch.Tensor
64     clean_rate: torch.Tensor
65     n_valid: int
66     n_rain: int
67
68
69 def supervision(logit: torch.Tensor, qlog: torch.Tensor, rate: torch.Tensor,
70                imerg_valid: torch.Tensor, yunnan_mask: torch.Tensor) -> Supervision:
71     occurrence_logits(logit)
72     same_shape_device("qlog spatial slice", qlog[:, :1] if isinstance(qlog, torch.Tensor) and qlog.ndim == 4 else qlog, logit)
73     quantiles(qlog)
74     same_shape_device("rate", rate, logit)
75     if rate.dtype != torch.float32 or rate.layout != torch.strided or rate.requires_grad:
76         raise ValueError("rate: nondifferentiable dense float32 IMERG reference required")
77     boolean_mask("imerg_valid", imerg_valid, logit)
78     boolean_mask("yunnan_mask", yunnan_mask, logit)
79     valid = imerg_valid & yunnan_mask
80     n_valid = int(valid.sum().item())
81     if n_valid == 0:
82         raise ZeroValidPixelsError("ZERO_VALID_SUPERVISED_YUNNAN_PIXELS; no silent skip")
83     if not bool(torch.isfinite(rate[valid]).all()) or bool((rate[valid] < 0).any()):
84         raise ValueError("Supervised valid precipitation must be finite nonnegative")
85     # Only an ephemeral arithmetic target is cleaned, exactly as in the old loss.
86     clean = torch.where(valid, rate, torch.zeros_like(rate))
87     rainy = clean > torch.tensor(.1, dtype=torch.float32, device=rate.device)
88     rainy_valid = rainy & valid # compare BEFORE converting rate to FP64
89     return Supervision(valid, rainy, rainy_valid, clean, n_valid, int(rainy_valid.sum().item()))

```

## 13.7 focal.py: 实际源码、函数位置与副本

来源 `src/yuntapr/experimental/phase_b_v2_ablations/focal.py`。

SHA256: 828189dfac57daf0520ffc8a37fb4b7608eccbd6154f3c04538049fc5313f94。参数语义见对应 `walkthrough`；下面是实际完整注释源码副本。

| 函数或方法（参数名）                                                              | 源码行 | 阅读入口                  |
|-------------------------------------------------------------------------|-----|-----------------------|
| <code>occurrence_numerator(logit, rainy, valid, *, alpha, gamma)</code> | 9   | 完整 type/default 见下面源码 |

```

1  """Stable candidate focal numerator, explicitly separate from normalization."""
2  from __future__ import annotations
3  import torch
4  from torch.nn import functional as F
5  from .config import finite_number
6  from .validation import boolean_mask, finite_tensor, graph_zero, occurrence_logits
7
8
9  def occurrence_numerator(logit: torch.Tensor, rainy: torch.Tensor, valid: torch.Tensor,
10                          *, alpha: float, gamma: float) -> torch.Tensor:
11      alpha, gamma = finite_number("alpha", alpha), finite_number("gamma", gamma)
12      if not 0 <= alpha <= 1 or gamma < 0:
13          raise ValueError("alpha in [0,1] and gamma>=0 required")
14      occurrence_logits(logit)
15      boolean_mask("rainy hard labels", rainy, logit)
16      boolean_mask("valid", valid, logit)
17      dtype = torch.float64 if logit.dtype == torch.float64 else torch.float32
18      with torch.autocast(logit.device.type, enabled=False):
19          if not bool(valid.any()):
20              return graph_zero(logit, dtype=dtype)
21          # FP16/BF16 inputs retain gradients through this cast; BCE computes FP32.
22          logits, labels = logit[valid].to(dtype), rainy[valid]
23          bce = F.binary_cross_entropy_with_logits(logits, labels.to(dtype), reduction="none")
24          alpha_t = torch.where(labels, torch.full_like(logits, alpha), torch.full_like(logits, 1 - alpha))
25          if gamma == 0:
26              terms = alpha_t * bce # deliberately .5*BCE for E1, not plain BCE
27          else:
28              pt = torch.exp(-bce)
29              terms = alpha_t * (1 - pt).pow(gamma) * bce
30          result = terms.sum()
31      finite_tensor("occurrence numerator", result)
32      return result

```

## 13.8 pinball.py: 实际源码、函数位置与副本

来源 `src/yuntapr/experimental/phase_b_v2_ablations/pinball.py`。

SHA256: `cc4c4c742a2ce5c465dc8c2eee786b85378d4d8845d7e065783f986bc264e25f`。参数语义见对应 `walkthrough`；下面是实际完整注释源码副本。

| 函数或方法（参数名）                                                                | 源码行 | 阅读入口                  |
|---------------------------------------------------------------------------|-----|-----------------------|
| <code>frozen_taus(device)</code>                                          | 7   | 完整 type/default 见下面源码 |
| <code>_pinball_from_errors(error, rainy_valid)</code>                     | 11  | 完整 type/default 见下面源码 |
| <code>conditional_pinball_numerator(qlog, log_target, rainy_valid)</code> | 32  | 完整 type/default 见下面源码 |

```

1  """FP64 conditional pinball in log1p(mm/h), fixed 32-tau mean then sum."""
2  from __future__ import annotations
3  import torch
4  from .validation import boolean_mask, finite_tensor, graph_zero, image_tensor, quantiles, same_shape_device
5
6
7  def frozen_taus(device: torch.device | str | None = None) -> torch.Tensor:
8      return (torch.arange(1, 33, device=device, dtype=torch.float64) - .5) / 32
9
10
11 def _pinball_from_errors(error: torch.Tensor, rainy_valid: torch.Tensor) -> torch.Tensor:
12     """Private arithmetic kernel: signed errors are NOT model quantiles.
13
14     This permits the P1-P4 hand-calculated constant-error fixtures without
15     pretending that constant qlog is a legal strictly ordered v2 prediction.

```

```

16     """
17     image_tensor("error", error, 32)
18     if error.dtype != torch.float64:
19         raise ValueError("Pinball signed errors require FP64")
20     boolean_mask("rainy_valid", rainy_valid, error[:, :1])
21     finite_tensor("pinball errors", error)
22     if not bool(rainy_valid.any()):
23         return graph_zero(error)
24     selected = error.movedim(1, -1)[rainy_valid.squeeze(1)] # [N_rain,32]
25     tau = frozen_taus(error.device)
26     per_pixel = torch.maximum(tau * selected, (tau - 1) * selected).mean(dim=-1)
27     result = per_pixel.sum()
28     finite_tensor("quantile numerator", result)
29     return result
30
31
32 def conditional_pinball_numerator(qlog: torch.Tensor, log_target: torch.Tensor,
33                                 rainy_valid: torch.Tensor) -> torch.Tensor:
34     quantiles(qlog)
35     same_shape_device("log_target", log_target, qlog[:, :1])
36     image_tensor("log_target", log_target, 1)
37     if log_target.dtype != torch.float64:
38         raise ValueError("log_target requires FP64")
39     boolean_mask("rainy_valid", rainy_valid, log_target)
40     finite_tensor("log_target", log_target)
41     if bool((log_target < 0).any()):
42         raise ValueError("log_target must be nonnegative logip(mm/h)")
43     threshold_log = torch.loglp(torch.tensor(.1, dtype=torch.float32, device=qlog.device).double())
44     if bool((log_target[rainy_valid] <= threshold_log).any()):
45         raise ValueError("rainy_valid cannot select a dry or threshold-equal reference target")
46     with torch.autocast(qlog.device.type, enabled=False):
47         # Only the explicitly declared singleton tau axis is expanded.
48         error = log_target - qlog
49         return _pinball_from_errors(error, rainy_valid)

```

### 13.9 total.py: 实际源码、函数位置与副本

来源 `src/yuntapr/experimental/phase_b_v2_ablations/total.py`。

SHA256: 9ce37393d2af2fbc762d198b2088cb4645d104d35f81b507c236bc3b39cf1ec1。参数语义见对应 walkthrough；下面是实际完整注释源码副本。

| 函数或方法（参数名）                                                                          | 源码行 | 阅读入口                  |
|-------------------------------------------------------------------------------------|-----|-----------------------|
| <code>combine_numerators(s_occ, s_qr, n_valid, *, lambda_q)</code>                  | 11  | 完整 type/default 见下面源码 |
| <code>candidate_loss(logit, qlog, rate, imerg_valid, yunnan_mask, *, config)</code> | 51  | 完整 type/default 见下面源码 |
| <code>scientific_conditional_pinball(self)</code>                                   | 46  | 完整 type/default 见下面源码 |

```

1  """Candidate training objective and unweighted components, no optimizer."""
2  from __future__ import annotations
3  from dataclasses import dataclass
4  import torch
5  from .config import AblationConfig, finite_number
6  from .focal import occurrence_numerator
7  from .pinball import conditional_pinball_numerator
8  from .validation import finite_tensor, supervision
9
10
11 def combine_numerators(s_occ: torch.Tensor, s_qr: torch.Tensor, n_valid: int,
12                       *, lambda_q: float) -> torch.Tensor:
13     weight = finite_number("lambda_q", lambda_q)

```

```

14     if weight <= 0 or type(n_valid) is not int or n_valid <= 0:
15         raise ValueError("Positive lambda_q and positive integer N_valid required")
16     for name, value in (("S_occ", s_occ), ("S_qr", s_qr)):
17         if not isinstance(value, torch.Tensor) or value.ndim != 0 or value.layout != torch.strided or value.dtype not in (torch.
            float32, torch.float64):
18             raise ValueError(f"{name}: FP32/FP64 scalar Tensor required")
19     if s_occ.device != s_qr.device:
20         raise ValueError("Numerator device mismatch")
21     if s_occ.device.type not in ("cpu", "cuda"):
22         raise ValueError("Numerator device must be CPU/CUDA")
23     for name, value in (("S_occ", s_occ), ("S_qr", s_qr)):
24         finite_tensor(name, value)
25         if bool(value < 0):
26             raise ValueError(f"{name}: loss numerator must be nonnegative")
27     result = (s_occ + weight * s_qr) / n_valid # weight exactly once
28     finite_tensor("weighted training objective", result)
29     return result
30
31
32 @dataclass(frozen=True)
33 class CandidateLossResult:
34     experiment_id: str
35     s_occ: torch.Tensor
36     s_qr: torch.Tensor
37     training_objective: torch.Tensor
38     weighted_quantile_per_valid: torch.Tensor
39     n_valid: int
40     n_rain: int
41     conditional_skipped: bool
42     status: str = "VALID_SYNTHETIC_OR_CANDIDATE_LOSS_CALL"
43     execution_authorization: bool = False
44
45     @property
46     def scientific_conditional_pinball(self) -> torch.Tensor | None:
47         # No lambda here. None explicitly means not estimable, not zero skill.
48         return self.s_qr / self.n_rain if self.n_rain else None
49
50
51 def candidate_loss(logit: torch.Tensor, qlog: torch.Tensor, rate: torch.Tensor,
52                  imerg_valid: torch.Tensor, yunnan_mask: torch.Tensor,
53                  *, config: AblationConfig) -> CandidateLossResult:
54     if type(config) is not AblationConfig:
55         raise ValueError("Explicit closed AblationConfig required")
56     config.__post_init__() # revalidate even if external code bypassed frozen setattr
57     target = supervision(logit, qlog, rate, imerg_valid, yunnan_mask)
58     s_occ = occurrence_numerator(logit, target.rainy, target.valid, alpha=config.alpha, gamma=config.gamma)
59     with torch.autocast(logit.device.type, enabled=False):
60         s_qr = conditional_pinball_numerator(qlog, torch.loglp(target.clean_rate.double()), target.rainy_valid)
61         total = combine_numerators(s_occ, s_qr, target.n_valid, lambda_q=config.lambda_q)
62         weighted = config.lambda_q * s_qr / target.n_valid
63     return CandidateLossResult(config.experiment_id, s_occ, s_qr, total, weighted,
64                               target.n_valid, target.n_rain, target.n_rain == 0)

```

### 13.10 metrics.py: 实际源码、函数位置与副本

来源 `src/yuntapr/experimental/phase_b_v2_ablations/metrics.py`。

SHA256: 3450edb2755339801a5fd55864091b1c22e0628fa49fce71bc5bff72811f9021。参数语义见对应 `walkthrough`；下面是实际完整注释源码副本。

| 函数或方法（参数名）                                                                       | 源码行 | 阅读入口                  |
|----------------------------------------------------------------------------------|-----|-----------------------|
| <code>common_validation_sums(logit, qlog, rate, imerg_valid, yunnan_mask)</code> | 38  | 完整 type/default 见下面源码 |
| <code>__post_init__(self)</code>                                                 | 17  | 完整 type/default 见下面源码 |

| 函数或方法（参数名）                               | 源码行 | 阅读入口                  |
|------------------------------------------|-----|-----------------------|
| <code>from_candidate(cls, result)</code> | 24  | 完整 type/default 见下面源码 |
| <code>conditional_pinball(self)</code>   | 28  | 完整 type/default 见下面源码 |
| <code>pooled(self, other)</code>         | 31  | 完整 type/default 见下面源码 |

```

1  """Unweighted reporting interface; common core always uses E0 in FP64."""
2  from __future__ import annotations
3  from dataclasses import dataclass
4  import math
5  import torch
6  from .config import get_config
7  from .total import CandidateLossResult, candidate_loss
8
9
10 @dataclass(frozen=True)
11 class ReportingSums:
12     s_occ: float
13     s_qr: float
14     n_valid: int
15     n_rain: int
16
17     def __post_init__(self) -> None:
18         if not all(type(x) in (float, int) and math.isfinite(x) and x >= 0 for x in (self.s_occ, self.s_qr)):
19             raise ValueError("Finite nonnegative unweighted reporting sums required")
20         if type(self.n_valid) is not int or type(self.n_rain) is not int or not 0 <= self.n_rain <= self.n_valid or self.n_valid <= 0:
21             raise ValueError("Positive N_valid and integer 0<=N_rain<=N_valid required")
22
23     @classmethod
24     def from_candidate(cls, result: CandidateLossResult) -> ReportingSums:
25         return cls(float(result.s_occ.detach()), float(result.s_qr.detach()), result.n_valid, result.n_rain)
26
27     @property
28     def conditional_pinball(self) -> float | None:
29         return self.s_qr / self.n_rain if self.n_rain else None
30
31     def pooled(self, other: ReportingSums) -> ReportingSums:
32         # Pool sums/counts rather than averaging unequal-batch metric values.
33         return ReportingSums(self.s_occ + other.s_occ, self.s_qr + other.s_qr,
34                               self.n_valid + other.n_valid, self.n_rain + other.n_rain)
35
36
37 @torch.no_grad()
38 def common_validation_sums(logit: torch.Tensor, qlog: torch.Tensor, rate: torch.Tensor,
39                             imerg_valid: torch.Tensor, yunnan_mask: torch.Tensor) -> ReportingSums:
40     """Fixed gamma2/lambda1 arithmetic, not an arm's weighted train objective.
41
42     Caller-supplied tensors only. This function does not run a model or read data.
43     The current tests call it exclusively with explicitly synthetic tensors.
44     """
45     result = candidate_loss(logit.double(), qlog, rate, imerg_valid, yunnan_mask, config=get_config("E0"))
46     return ReportingSums.from_candidate(result)

```

### 13.11 readiness.py: 实际源码、函数位置与副本

来源 `src/yuntapr/experimental/phase_b_v2_ablations/readiness.py`。

SHA256: 2f6ca9f1f5cf0da458cc008e05bb24d4c1ecd776172f35bf1b8cef81f23492e3。参数语义见对应 `walkthrough`；下面是实际完整注释源码副本。

| 函数或方法（参数名）                                                                         | 源码行 | 阅读入口                  |
|------------------------------------------------------------------------------------|-----|-----------------------|
| require_sha(value, name)                                                           | 17  | 完整 type/default 见下面源码 |
| state_metadata_digest(tensors)                                                     | 36  | 完整 type/default 见下面源码 |
| audit_fresh_metadata(records)                                                      | 68  | 完整 type/default 见下面源码 |
| review_checkpoint_<br>binding(metadata, expected, *,<br>resume_approval_reference) | 99  | 完整 type/default 见下面源码 |
| __post_init__(self)                                                                | 28  | 完整 type/default 见下面源码 |
| validate(self)                                                                     | 56  | 完整 type/default 见下面源码 |
| describe(self)                                                                     | 140 | 完整 type/default 见下面源码 |
| run(self, *args, **kwargs)                                                         | 147 | 完整 type/default 见下面源码 |

```

1  """Metadata-only identity checks and an unconditionally blocked runner stub.
2
3  No torch import, tensor state loading, dataset access or approval issuance.
4  Synthetically valid metadata never implies that real initial states exist.
5  """
6  from __future__ import annotations
7  from dataclasses import asdict, dataclass
8  import hashlib
9  import json
10 import re
11 from typing import NoReturn, Sequence
12 from .config import RunSpec, proposed_runs, workload
13
14 INPUT_DIFFERENCES = {"backbone.enc0.conv1.weight", "backbone.enc0.skip.weight"}
15
16
17 def require_sha(value: object, name: str) -> None:
18     if type(value) is not str or re.fullmatch(r"[0-9a-f]{64}", value) is None:
19         raise ValueError(f"{name}: lowercase SHA256 required, never a path or credential")
20
21
22 @dataclass(frozen=True)
23 class TensorIdentity:
24     shape: tuple[int, ...]
25     dtype: str
26     sha256: str
27
28     def __post_init__(self) -> None:
29         if type(self.shape) is not tuple or not self.shape or not all(type(n) is int and n > 0 for n in self.shape):
30             raise ValueError("Nonempty positive integer shape required")
31         if self.dtype != "float32":
32             raise ValueError("Frozen candidate parameter dtype is float32")
33         require_sha(self.sha256, "tensor identity")
34
35
36 def state_metadata_digest(tensors: dict[str, TensorIdentity]) -> str:
37     if type(tensors) is not dict or not tensors or not all(type(k) is str and k and
38         type(v) is TensorIdentity for k, v in tensors.items()):
39         raise ValueError("Named tensor identity metadata required")
40     for value in tensors.values():
41         value.__post_init__()
42     body = json.dumps({k: asdict(v) for k, v in sorted(tensors.items())}, sort_keys=True, separators=(",", ":"))
43     return hashlib.sha256(body.encode()).hexdigest()
44
45
46 @dataclass(frozen=True)
47 class FreshRecord:
48     run: RunSpec
49     tensors: dict[str, TensorIdentity]
50     state_sha256: str
51     post_pair_rng_sha256: str

```

```

52 epoch_order_sha256: str
53 initializer_code_sha256: str
54 historical_checkpoint_loaded: bool = False
55
56 def validate(self) -> None:
57     if type(self.run) is not RunSpec:
58         raise ValueError("Closed candidate RunSpec required")
59     self.run.__post_init__()
60     for name in ("state_sha256", "post_pair_rng_sha256", "epoch_order_sha256", "initializer_code_sha256"):
61         require_sha(getattr(self, name), name)
62     if self.historical_checkpoint_loaded is not False:
63         raise ValueError("Historical model/optimizer/RNG state transfer forbidden")
64     if state_metadata_digest(self.tensors) != self.state_sha256:
65         raise ValueError("Declared state digest differs from supplied tensor metadata")
66
67 def audit_fresh_metadata(records: Sequence[FreshRecord]) -> dict:
68     """Review declarations only; cannot prove origins or instantiate a model."""
69     if len(records) != 18 or not all(type(r) is FreshRecord for r in records):
70         raise ValueError("Exactly 18 fresh metadata records required")
71     for r in records:
72         r.validate()
73     table = {r.run.run_id: r for r in records}
74     if set(table) != {r.run_id for r in proposed_runs()}:
75         raise ValueError("Missing, duplicate or unregistered run identity")
76     if len({r.initializer_code_sha256 for r in records}) != 1:
77         raise ValueError("Initializer code identity differs across arms")
78     for seed in (2026, 2027, 2028):
79         subset = [r for r in records if r.run.seed == seed]
80         if len({r.post_pair_rng_sha256 for r in subset}) != 1 or len({r.epoch_order_sha256 for r in subset}) != 1:
81             raise ValueError("Same-seed RNG/order metadata differs")
82         for model in ("B0_MATCHED_V2", "B1_V2"):
83             arms = [r for r in subset if r.run.model == model]
84             if len({r.state_sha256 for r in arms}) != 1:
85                 raise ValueError("Same-model same-seed full fresh states differ across arms")
86             a = table[RunSpec("EO", "B0_MATCHED_V2", seed).run_id].tensors
87             b = table[RunSpec("EO", "B1_V2", seed).run_id].tensors
88             if a.keys() != b.keys():
89                 raise ValueError("B0/B1 parameter names differ")
90             different = {k for k in a if a[k].shape != b[k].shape}
91             if different != INPUT_DIFFERENCES or any(a[k] != b[k] for k in a if k not in different):
92                 raise ValueError("Only two frozen input-shape differences permitted; shared tensors must match")
93     return {"metadata_consistent": True, "actual_model_instantiated": False,
94           "actual_initialization_proven": False, "scope": "DECLARED_OR_SYNTHETIC_METADATA_ONLY",
95           "can_launch_formal_training": False}
96
97
98
99 def review_checkpoint_binding(metadata: dict, expected: dict, *, resume_approval_reference: str | None = None) -> dict:
100     """No files are opened and no state can be applied by this review.
101
102     A supplied reference is not authenticated here and cannot authorize resume.
103     Future formal integration must bind a real independent event to LAST SHA.
104     """
105     fields = {"run_id", "last_sha256", "model_sha256", "optimizer_sha256", "rng_sha256",
106              "scheduler_sha256", "code_sha256", "protocol_sha256", "data_sha256", "init_sha256",
107              "completed_epoch", "retained_updates", "boundary", "original_execution_reference"}
108     if type(metadata) is not dict or set(metadata) != fields or type(expected) is not dict:
109         raise ValueError("Closed checkpoint metadata required")
110     if metadata["run_id"] not in {r.run_id for r in proposed_runs()}:
111         raise ValueError("Checkpoint run identity is not in candidate matrix")
112     for key in fields:
113         if key.endswith("sha256"):
114             require_sha(metadata[key], key)
115     epoch, updates = metadata["completed_epoch"], metadata["retained_updates"]
116     if type(epoch) is not int or not 1 <= epoch <= 9 or type(updates) is not int or updates != epoch * 5228:
117         raise ValueError("Complete-epoch retained budget mismatch")
118     if metadata["boundary"] != "COMPLETED_TRAIN_AND_VALIDATION_EPOCH_ONLY":
119         raise ValueError("Partial epoch is not a legal LAST")
120     if type(metadata["original_execution_reference"]) is not str or not metadata["original_execution_reference"]:
121         raise ValueError("Original execution provenance reference missing")

```

```

122     binding = {"run_id", "last_sha256", "code_sha256", "protocol_sha256", "data_sha256", "init_sha256"}
123     if set(expected) != binding or any(metadata[k] != expected[k] for k in binding):
124         raise ValueError("Expected LAST/code/protocol/data/init/run binding differs")
125     if resume_approval_reference is not None and (type(resume_approval_reference) is not str or not resume_approval_reference):
126         raise ValueError("Nonempty independent-event reference or None required")
127     return {"metadata_consistent": True, "remaining_retained_updates": 47052 - updates,
128            "next_epoch": epoch + 1 if epoch < 9 else None, "approval_authenticity_verified": False,
129            "resume_approval_reference_present": resume_approval_reference is not None,
130            "can_apply_checkpoint_state": False, "can_launch_formal_training": False}
131
132
133 class FormalExecutionBlocked(RuntimeError):
134     pass
135
136
137 class BlockedRunner:
138     """Design skeleton with no execution implementation, even after flag edits."""
139
140     def describe(self) -> dict:
141         return {"runs": [r.run_id for r in proposed_runs()], "budget": workload(),
142                "runner_implemented": False, "can_launch_formal_training": False,
143                "required": ["Independent scientific approval", "Researcher code review",
144                           "Explicit formal integration approval", "Data and compute permissions",
145                           "Real fresh/data/code identities", "Separately bound stage execution authorization"]}
146
147     def run(self, *args: object, **kwargs: object) -> NoReturn:
148         raise FormalExecutionBlocked("Candidate skeleton has NO TRAINING CAPABILITY; new reviewed runner and independent
        approvals required")

```

## 14 关键数学、解析梯度与源码对应

$v = m_{\text{IMERG}} \wedge m_{\text{Yunnan}}$ ,  $z = \mathbf{1}\{y_{32} > \text{float32}(0.1)\}$ ,  $N_v = \sum v$ ,  $N_r = \sum vz$ ; 比较在提升精度前完成。

$$\begin{aligned} b(l, z) &= \max(l, 0) - lz + \log(1 + e^{-|l|}), & p_t &= e^{-b}, \\ \alpha_z &= \alpha z + (1 - \alpha)(1 - z), & S_o &= \sum v \alpha_z (1 - p_t)^\gamma b, \\ \tau_i &= (i - 0.5)/32, & e_i &= \log(1 + y) - q_i, \\ \rho_{\tau_i}(e_i) &= \max(\tau_i e_i, (\tau_i - 1)e_i), & S_q &= \sum v z \frac{1}{32} \sum_i \rho_{\tau_i}(e_i), \\ L_{\text{train}} &= (S_o + \lambda_q S_q)/N_v, & \text{CPB} &= S_q/N_r. \end{aligned}$$

Focal 对应 focal.py 中的 BCEWithLogits、exp、pow、sum; Pinball 对应 pinball.py 中的固定 tau、maximum、mean、sum; total.py 只乘一次 lambda。 $\gamma = 0, \alpha = .5$  仍为 .5 BCE。CPB 不带权重, 无雨不可估计。

$$\frac{\partial S_o}{\partial l} = \alpha_z(p - z)\{m^\gamma + \gamma b p_t m^{\gamma-1}\}, \quad m = 1 - p_t, \quad \gamma > 0; \quad \gamma = 0: \alpha_z(p - z).$$

$$\frac{\partial L}{\partial q_i} = \frac{\lambda_q}{32N_v} \begin{cases} -\tau_i, & e_i > 0, \\ 1 - \tau_i, & e_i < 0. \end{cases}$$

以上梯度仅在有效位置并按相应雨标签生效;  $e = 0$  为折点。test\_gradients.py 独立核对解析式、FP64 差分和全部 32tau; 不是实际模型性能结论。

$$\eta(u) = \begin{cases} 10^{-4}u/5228, & 1 \leq u \leq 5228, \\ 10^{-6} + (10^{-4} - 10^{-6}) \frac{1 + \cos(\pi(u - 5228)/(261400 - 5228))}{2}, & 5228 < u \leq 47052. \end{cases}$$

config.py 的 learning\_rate\_prefix 逐位置与冻结纯数学函数对照; 没有 optimizer 或正式更新。BlockedRunner 只抛错。V2\_PHASE\_B\_AUTHORIZED=false, 历史恢复 NOT\_GRANTED, 2025 访问 0。工程交付到此停止。